Django Readiness Challenge Set — Solution Guide

This document contains conceptual solution outlines and architectural guidance for the Django Readiness Challenge Set. The goal is not to provide copy-paste implementations, but to demonstrate the reasoning, backend structure, database thinking, validation strategy, and object-oriented design expected before beginning Django development seriously.

1. School Management System (Console-Based)

Core Classes

- Create Student, Teacher, and Course classes.
- Use constructors (__init__) to initialize attributes.
- Store enrollments using lists or dictionaries.
- Use methods like register_student(), assign_course(), and enroll_student().

Data Relationships

- One teacher can teach many courses.
- A student can register for many courses.
- A course can contain many students.
- Student ↔ Course is a Many-to-Many relationship.

Validation Logic

- Reject duplicate emails.
- Reject empty names and invalid ages.
- Validate course codes before enrollment.

File Handling

- Use text or JSON files to store student, teacher, and course records.
- Load records during program startup.
- Append new records without overwriting existing data.

SQL Thinking

- INSERT INTO students(fullname, age, email) VALUES (...);
- SELECT * FROM students WHERE id = 1;
- UPDATE courses SET title = 'Python' WHERE id = 2;
- DELETE FROM teachers WHERE id = 4;

HTTP Concept

- Student registration should use POST because data is being submitted securely.
- GET should only retrieve data.

2. Hospital Patient Record System

OOP Structure

- Create a base Person class.
- Patient and Doctor should inherit from Person.
- Use inheritance to avoid duplicate attributes.

Appointment Logic

- Check doctor availability before booking.
- Prevent duplicate appointments at the same time.

Billing Logic

- Total Bill = Consultation Fee + Medication Fee.
- Store bill history for future retrieval.

Database Relationships

- One doctor can have many patients.
- One patient can have many appointments.
- Doctors and patients may form Many-to-Many relationships through appointments.

Exception Handling

- Handle invalid dates using try/except.
- Handle missing patient records gracefully.

Request Flow

- Browser sends request → server validates data → database stores records → response is returned to browser.

3. E-Commerce Inventory & Order System

Core Classes

- Create User, Customer, Admin, Product, Cart, and Order classes.

- Use inheritance for Customer and Admin from User.

Authentication Logic

- Validate usernames and passwords.
- Reject duplicate usernames.
- Store hashed passwords in real systems.

Cart Logic

- Allow multiple products in cart.
- Automatically calculate totals using loops.

Stock Validation

- Prevent users from buying products with zero stock.
- Reduce stock after successful order placement.

SQL Thinking

- SELECT * FROM products;
- UPDATE products SET stock = stock - 1 WHERE id = 3;
- INSERT INTO orders(customer_id, total) VALUES (...);

Session Thinking

- Sessions help the server remember logged-in users between requests.

4. Banking/ATM System

Core Banking Logic

- Create Account and Transaction classes.
- Each account should contain balance and transaction history.

Security Logic

- Prevent withdrawals greater than balance.
- Reject duplicate account numbers.
- Validate PIN before transactions.

Encapsulation

- Protect sensitive fields like balance using methods instead of direct modification.

Transaction Thinking

- Money transfer must debit sender and credit receiver together.
- If one operation fails, rollback should occur.

SQL Examples

- UPDATE accounts SET balance = balance - 500 WHERE id = 1;
- UPDATE accounts SET balance = balance + 500 WHERE id = 2;
- INSERT INTO transactions(account_id, amount) VALUES (...);

HTTP Concepts

- Transfers should use POST because they modify server data.

5. Student Result Processing Portal

Core Classes

- Create Student, Lecturer, Course, and Result classes.
- Use methods for GPA calculation.

GPA Logic

- Multiply grade points by course units.
- Divide total grade points by total units.

Relationships

- One lecturer can teach many courses.
- Students can register for many courses.
- Results connect students and courses.

Validation

- Reject scores below 0 or above 100.
- Prevent duplicate course registration.

Authentication Flow

- Student logs in → credentials validated → session created → access granted.

SQL Thinking

- SELECT * FROM results WHERE student_id = 2;

- UPDATE results SET score = 75 WHERE id = 5;
- DELETE FROM results WHERE id = 7;

A learner who understands and can independently implement most of these ideas is usually ready to begin Django development seriously. The key indicator is not memorization, but the ability to model real systems, reason about data flow, validate input, and debug problems logically.